

保守性重視の開発実践

勉強会

SE保守・運用職 → SE開発職 への転向を目指す方へ

| 本日のアジェンダ

時間	セクション	形式
0:00～0:20	オリエンテーション	講義
0:20～0:50	リファクタリング基礎	講義
0:50～1:30	個人演習	ハンズオン
1:30～2:05	ペアレビュー	グループワーク
2:05～2:30	全体共有・解説	発表＋講義
2:30～2:50	変更影響ケーススタディ	演習
2:50～3:00	まとめ・質疑応答	講義

SE保守・運用 vs SE開発

観点	SE保守・運用	SE開発
主な仕事	監視・障害対応・パッチ適用	新機能の設計・実装・テスト
コードとの向き合い方	動いているコードを壊さない	将来変えやすいコードを書く
変更の粒度	最小限の変更（リスク回避）	積極的な改善（品質向上）
評価軸	障害件数・稼働率	開発速度・コード品質

転向で最も問われるスキル

「動けばいい」から「変えやすく・読みやすく・テストしやすい」への思考転換

なぜ保守性・拡張性が重要か？

ソフトウェアのコストの **70～80%** はリリース後のメンテナンスコスト

読みにくいコード

- 修正に時間がかかる
- バグを埋め込む
 - 障害
 - 信頼失墜

保守性の高いコードがもたらす効果

- ・ 新メンバーがすぐに理解できる（オンボーディングコスト削減）
- ・ バグの影響範囲が限定される（障害の局所化）
- ・ 仕様変更に対応できる（ビジネス価値向上）

リファクタリングとは？

外部から見た振る舞いを変えずに、内部構造を改善すること

	内容
✔ やること	コードの整理・命名改善・重複排除・責務分離
✖ やらないこと	新機能追加・バグ修正（別タスク）

リファクタリングのタイミング

- 1. 新機能追加の 前（理解しやすくしてから追加する）
- 2. バグ修正の 後（根本原因を潰してから整理する）
- 3. コードレビューで指摘を受けたとき

DRY原則

"Don't Repeat Yourself"

すべての知識はシステム内で 単一・明確・権威ある表現 を持つべき

DRY違反の例 — 同じ顧客取得ロジックが3箇所に散在

```
def process_order(cid):  
    if cid == 1:  
        name = "田中太郎" # ← 重複  
        email = "tanaka@example.com"  
  
def process_return(cid):  
    if cid == 1: # ← まったく同じロジックが再登場  
        name = "田中太郎"  
        email = "tanaka@example.com"
```

問題: 顧客データを修正する際に全箇所を探して直す必要がある → 修正漏れでバグ発生

| DRY原則 — 改善後

単一化した例

```
CUSTOMERS = {  
    1: {"name": "田中太郎", "email": "tanaka@example.com"},  
    2: {"name": "鈴木花子", "email": "suzuki@example.com"},  
}  
  
def get_customer(customer_id: int) -> dict | None:  
    return CUSTOMERS.get(customer_id)  
  
# 以降はどこでも get_customer() を呼ぶだけ  
def process_order(cid):  
    customer = get_customer(cid)  
    ...
```

顧客データの変更は 1箇所だけ 修正すればOK

SOLID原則

略称	原則名	一言まとめ
S	単一責任の原則	「1つのことだけやる」
O	開放閉鎖の原則	「既存コードを変えずに機能追加できる」
L	リスコフの置換原則	「サブクラスは親の約束を守る」
I	インタフェース分離の原則	「使わないメソッドを押しつけない」
D	依存性逆転の原則	「抽象に依存する」

本日の演習で特に意識してほしい原則

→ S（単一責任） と O（開放閉鎖）

SOLID全体は「設計を見るための地図」として扱い、演習では変更しやすさに直結する **S/O** に絞って練習します。

SRP違反の例

1つの関数が5つの責務を持っている

```
# process_order が全部やっている
def process_order(customer_id, items, discount_code):
    total = sum(...)           # ① 合計計算
    if discount_code == "GOLD10":
        total *= 0.9           # ② 割引適用
    if total < 3000:
        shipping = 500         # ③ 送料計算
    print(f"メール送信先: {email}") # ④ メール送信
    for item in items:
        print(f"在庫を減らします") # ⑤ 在庫更新
```

変更理由が5つある = 変更のたびに全体への影響を確認しなければならない

命名規則のベストプラクティス

意図が伝わる名前を使う

✗ NG	✓ OK	理由
c	customer	省略は読む速度を下げる
t	subtotal	何のtotalか不明
ft	final_total	2単語で意味が明確に
dc	discount_code	コンテキストなしでは意味不明

良いコメントとは「なぜ」を書くもの

✗ NG: コードを読めばわかることを書く
total = total * 0.9 # totalに0.9をかける

✓ OK: 「なぜ」を書く
total = total * 0.9 # ゴールド会員は10%割引（2026年料金規程§3.2）

コード臭（Code Smell）の典型例

Code Smell	説明	対処法
重複コード	同じロジックが複数箇所にある	関数・定数として抽出
長すぎる関数	1関数が100行を超える	責務ごとに分割
意味不明な命名	a , tmp , data2 など	意図が伝わる名前に
マジックナンバー	意味不明な数値が直書き	名前付き定数にする
コメントアウトコード	使わないコードが残っている	削除する（git履歴に残る）
条件文の爆発	if/elif が10個以上連なる	辞書・ポリモーフィズムで置換

リファクタリング手順

安全に進めるための5ステップ

1. テストを書く — 変更前の動作を保証する
2. 小さく変更する — 一度に大きく変えない
3. 動作確認する — テストを実行して壊れていないか確認
4. コミットする — 変更履歴を細かく残す
5. 繰り返す

「テストなきリファクタリングはリスクでしかない」
— Martin Fowler, *Refactoring* より

個人演習（40分）

02_演習_問題コード.py を開いてください

演習の進め方

1. コードを読み、問題点をすべてリストアップする（15分）
2. 問題点を優先順位付けし、改善方針を決める（10分）
3. Googleドキュメント上で、改善後のコードに書き換える（15分）

観点チェックリスト

- ☐ DRY原則に違反した重複コードはどこか？
- ☐ 1つの関数が複数の責務を持っていないか？（SRP）
- ☐ 変数名・関数名は意図を正確に伝えているか？
- ☐マジックナンバーが直書きされていないか？
- ☐ エラーケース（存在しないIDなど）が考慮されているか？
- ☐ 分離した関数は単体テストしやすい形になっているか？

ペアレビュー（35分）

進め方

1. 相手のGoogleドキュメントを開く（5分）
2. コメント機能で良い点・改善提案を記入する（20分）
3. お互いのコメントを読み合わせ・議論する（10分）

良いレビューコメントの例

- ✓ 良い点: `get_customer()` として抽出したことで DRY 原則が守られています。
将来的に顧客データをDBから取得する変更も1箇所の修正で済みます。
- 💡 改善提案: `calculate_subtotal()` の中でカテゴリごとに条件分岐していますが、
全カテゴリで同じ計算をしているため条件分岐は不要では？

レビューはコードを改善するもの。書いた人を評価するものではない。

模範解答のポイント

変更点	効果
顧客取得を <code>get_customer()</code> に集約	DRY原則。顧客データ変更が1箇所で完結
小計計算を <code>calculate_subtotal()</code> に分離	SRP。テスト可能な単位に分割
送料ルールを定数テーブルに	OCP。新しいルール追加が既存コード無修正で可能
割引率を辞書に	条件分岐の爆発を解消
メール送信を <code>send_email()</code> に集約	DRY。メール形式変更が1箇所で完結
<code>None</code> チェックを早期リターンに	ネストを浅く保つ（ガード節パターン）

ケース：仕様変更依頼

依頼: 「割引コード `NEW30` (30%OFF) を追加してほしい。
また、送料無料ラインを5,000円から8,000円に引き上げてほしい。」

変更影響の考え方

`04_変更影響チェックリスト.md` を開き、Step 1～5を記入しながら進めます。

1. 変更する対象を特定する → どの関数・クラス・定数が直接変わるか？
2. 呼び出し元（上流）を調べる → その関数を呼んでいるのはどこか？
3. 呼び出し先（下流）を調べる → その関数が依存しているものは何か？
4. テストへの影響を確認する → どのテストケースが失敗するか？
5. 外部インタフェースへの影響を確認する → API仕様・ドキュメントは？

リファクタリング前後の変更コスト比較

リファクタリング前（問題コード）

- 割引コード追加: `if/elif` チェーンに1行追加
- 送料ルール変更: マジックナンバーを探して修正 → 見落としリスクあり

リファクタリング後（模範解答）

```
# 割引コード追加: 辞書に1行追加するだけ
DISCOUNT_RATES = {
    "GOLD10": 0.10,
    "SILVER5": 0.05,
    "NEW30": 0.30, # ← これだけ
}

# 送料ルール変更: 定数テーブルの数値を1箇所変えるだけ
SHIPPING_RULES = [
    (8000, 0), # ← 5000 → 8000 に変更
    (3000, 300),
    (0, 500),
]
```

今後の学習ロードマップ

【現在地】 保守性の概念・リファクタリングの基礎を理解した

↓ 1～2ヶ月

【ステップ1】 実務コードのリファクタリング体験

既存プロジェクトのコードで Code Smell を1つ見つけ、改善する

推奨書籍：「リファクタリング」Martin Fowler著

↓ 2～3ヶ月

【ステップ2】 テストを書く習慣

単体テスト (pytest) の基礎を学ぶ

TDD (テスト駆動開発) の基本サイクルを体験する

↓ 3～6ヶ月

【ステップ3】 設計パターンの習得

GoFデザインパターン (Strategy・Factory など)

クリーンアーキテクチャの考え方

参考資料

書籍・資料	難易度	おすすめポイント
「リファクタリング 第2版」 Martin Fowler	★★★	リファクタリングのバイブル。具体的な手法が68種類
「Clean Code」 Robert C. Martin	★★★	命名・関数設計の哲学が体系的にまとまっている
「良いコードを書く技術」 (技術評論社)	★★	日本語で読みやすい入門書

ご参加ありがとうございました

「動けばいい」から
「変えやすく・読みやすく・テストしやすい」へ
お疲れ様でした！